LODC

Ley Orgánica de Defensa del Consumidor · Ecuador

# Asistente inteligente generador de reclamos de consumo


**Integrantes**

Moisés Omar Loor Vasquez · Mateo Sebastian Obregon Meza · Fausto Fernando Gavilanes Procel


**Backend** https://asistente-inteligente-4kiq.onrender.com

**Frontend** https://generadoreclamos.vercel.app/


**Seminario de IA Generativa**

UNIANDES · Grupo 2

01 · PROBLEMA Y USUARIO

# Derechos claros, reclamos que nunca llegan

**Fuente:** docs/01-propuesta.md

## Problema

- Garantías incumplidas, cobros indebidos, publicidad engañosa, compras por internet.
- La LODC reconoce derechos, pero no se sabe cuál aplica, qué reunir, qué camino seguir ni cómo redactar el reclamo.
- Resultado: reclamos legítimos que no se presentan, o se archivan por falta de sustento.
- Hoy: redes sociales, consejos contradictorios, escritos sin citar la norma, o desiste.

## Usuarios

- **Principal:** consumidor sin formación jurídica. Necesita saber si su caso tiene respaldo en la LODC y un reclamo listo para presentar.
- **Secundario:** consumidor con poca experiencia digital. Necesita guía paso a paso con preguntas fáciles.

## Qué hace la app

- Filtra relatos ajenos al consumo; pide aclaración si el relato es insuficiente.
- Clasifica el reclamo (3 tipos) por búsqueda vectorial sobre los artículos de la LODC.
- Cita artículos con metadatos del corpus; no los inventa.
- Sin base normativa suficiente: se abstiene o pide aclaración.

## Qué NO hace

- No es asesoría legal ni predice resultados o indemnizaciones.
- No presenta el reclamo ante ninguna entidad: solo lo prepara.
- Nada fuera de la LODC: laboral, penal, arrendamientos, tributaria, familiar.
- No inventa artículos.
- No envía al modelo los datos de identificación del consumidor ni del proveedor.

*«Orientamos al consumidor con base en la ley citada, y cuando no hay base, nos abstenemos».*

03 · ARQUITECTURA

# El modelo interpreta; Java decide y calcula Fuente

| | |
|---|---|
| Entrada | Portero semántico que descarta lo ajeno a la ley de consumo, es como el filtro que determina si un relato tiene coherencia y tiene que ver con la LODC. |
| 1 · Clasificación | Traduce el relato coloquial a consulta jurídica (la LODC usa lenguaje formal), busca fragmentos en Qdrant y un paso determinista infiere el tipo (1 de 3) por metadatos. Si no alcanza: aclaración o abstención. |
| 2 · Recolección | Un agente pre-llena fechas, montos y proveedor; el sistema detecta faltantes, pregunta una a una y valida cada respuesta. |
| 3 · Viabilidad | Compuerta de la Defensoría del Pueblo, razonamiento con artículos LODC, informe que el usuario acepta (human-in-the-loop), rutas de acción y plan. |
| 4 · Cálculos | El modelo no calcula: catálogo por tipo → artículo → función Java (BigDecimal, java.time): plazos, multa mínima, interés legal. Lo de autoridad: A_DEFINIR_POR_AUTORIDAD. |
| 5 · Documento | El modelo redacta solo hechos y pretensiones; el código inyecta datos, artículos y cuantía en la plantilla .docx fija; el usuario descarga. |

### Front

React + TypeScript. Diseño: Fausto; consumo del API: Mateo. Fausto dejó herramientas e info base del backend.

### Servicios

LLM externo (cliente compatible OpenAI), Qdrant Cloud, Redis (sesión y límite de tasa), PostgreSQL, Docker en Render. Estado en Redis, persistible en PostgreSQL.

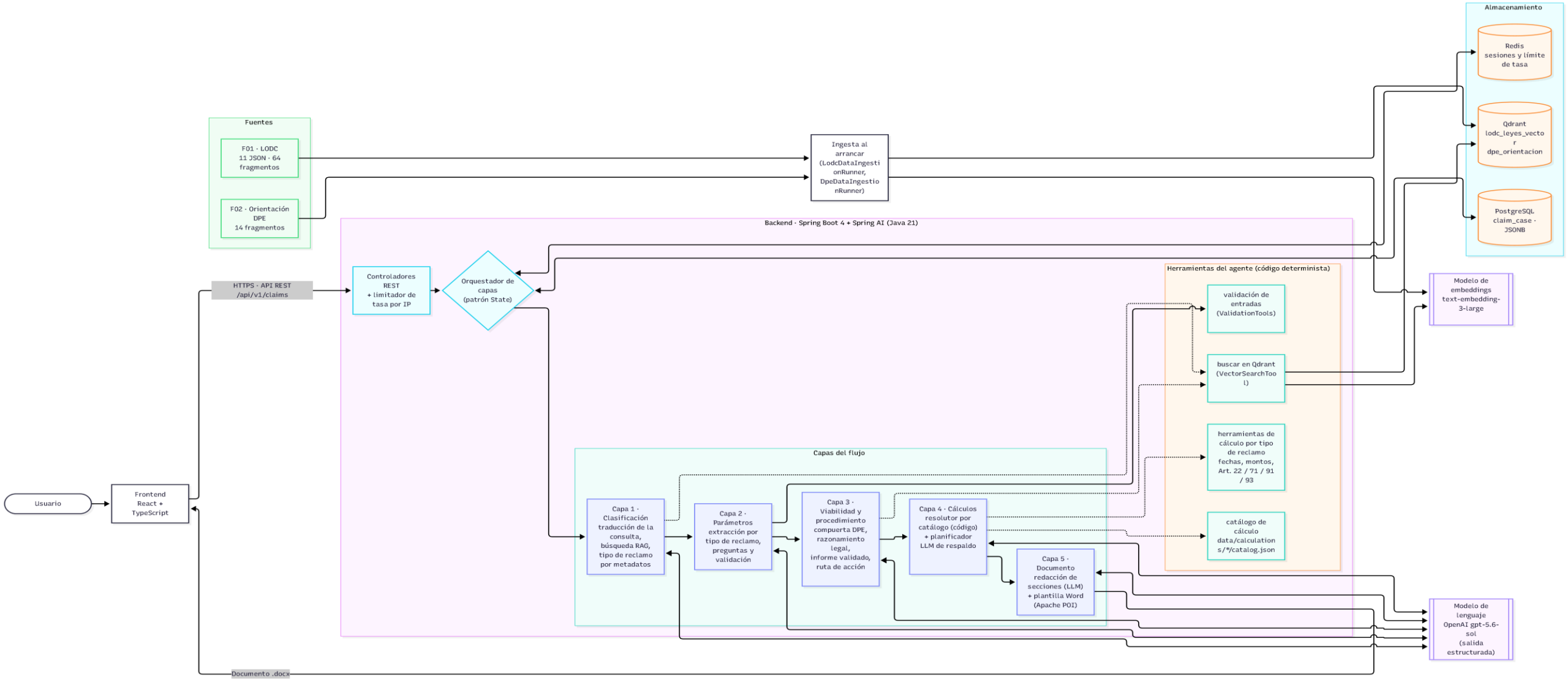
### Corpus

LODC 64/64 artículos verificados contra el PDF oficial, con manifiesto de integridad. Recuperación 0,30 · abstención 0,40.

### Garantías

Artículos desde metadatos, no generación libre. Datos de identificación fuera del modelo, inyectados por código. ADR: docs/09-decisiones.md D8–D12.

# Arquitectura-del-flujo agentico



# Pasan 19 de 20;

**Fuentes:** `docs/08-pruebas.md`, `eval/20_relatos_referencia/resultados_url_publica_2026-11-07.csv`.

**19/20**

casos pasan contra la URL pública (07-nov-2026)

## A · Pruebas funcionales

| Resultado | Cifra |
|---|---|
| **Casos que pasan** | 19 de 20 |
| **Fallo** | 1: C09, faltó art. 16 (tipo correcto) |
| **Clasificación (12 reclamos)** | Tipo 12/12 · artículos 11/12 |
| **Abstención (4)** | 4/4 |
| **Fuera de dominio + inyección (4)** | 4/4 |

## B · Evaluación final vs semana 3

| Corrida | Tipo | Arts. | Abst. | Rech. |
|---|---|---|---|---|
| **Semana 3*** | 12/12 | — | 1/4 | 3/4 |
| **04-11 local** | 11/12 | 11/12 | 4/4 | 4/4 |
| **07-11 local** | 12/12 | 12/12 | 4/4 | 4/4 |
| **07-11 URL pública** | 12/12 | 11/12 | 4/4 | 4/4 |

Mejora principal: abstención y rechazo (1/4 y 3/4 → 4/4). El equipo la atribuye al umbral de abstención y al portero; no se aisló el efecto de cada uno.

**Matices (no ocultar):** n = 20: p95 ≈ máximo, indicativo · una máquina, una corrida, varía con la carga del proveedor · arranque en frío ~80 s excluido · latencia en corrida aparte de la funcional (misma fecha y casos) · solo /start (capa 1), capas 2–5 sin medir · los 400 son el portero, no errores · límite 10/min por IP → peticiones cada ~7 s.

05 · SEGURIDAD

# Claves fuera del código, errores con sentido

*«Las claves nunca están en el código, y si el modelo falla el usuario ve un mensaje claro, nunca el error del proveedor».*

## 1 · Dónde viven las claves

- Solo en Render → Environment: LLM_API_KEY, QDRANT_API_KEY, REDIS_PASSWORD, DB_USER, DB_PASSWORD, DB_URL (captura con valores ocultos).
- Sin valores por defecto: si falta una variable, la app no arranca.
- .env en .gitignore y fuera de la imagen Docker; contenedor con usuario sin privilegios (USER app).

## 2 · Si el modelo falla

- Timeout de conexión y lectura: 5 s (HttpClientConfig).
- Reintentos con backoff: hasta 4, inicial 2 s, ×2, máx. 10 s. Solo ante 429/500/502/503/504; nunca ante 400/401/403/404.
- Agotados: 429 «proveedor saturado…», 504 «no se pudo conectar…», 502 «rechazó la solicitud». Detalle técnico solo en el log.
- Verificado leyendo el código; no se provocó un fallo real en producción.

## 3 · Límite de tasa

- 10 solicitudes / 60 s por IP: Sliding Window Counter en script Lua sobre Redis (atómico).
- Evidencia real: HTTP 429 con Retry-After: 46 en la petición 12 (13 en otra corrida; ±2, ventana aproximada).
- Si Redis cae: 503 y no deja pasar (fail-closed).

## 4 · Protección del contenido

- Portero semántico: I19 (pedir el system prompt) rechazada; I20 (instrucción maliciosa en reclamo válido) pidió aclaración, no obedeció.
- Abstención por similitud < 0,40 (segunda barrera, con prueba unitaria).
- Artículos desde metadatos del corpus, no generación libre.

06 · LIMITACIONES

# Lo que nunca hará, y lo que aún falta

## A · Lo que NUNCA hará (alcance negativo) y cómo se hace cumplir

| Límite por diseño | Cómo se hace cumplir | Evidencia |
|---|---|---|
| **No es asesoría legal: orienta, no reemplaza a un abogado ni garantiza el resultado** | Aviso de IA en el README, tono orientativo, derivación a la Defensoría del Pueblo (riesgo R3) | README, s4g_s4h_aviso_ia_y_readme.md |
| **No predice resultados ni indemnizaciones (ni daños y perjuicios)** | Los cálculos los hace código Java | docs/03-arquitectura.md, capa 4 |
| **No gestiona trámites: no presenta ni envía el reclamo** | Solo genera un .docx descargable desde plantilla fija | docs/03-arquitectura.md, capa 5 |
| **Nada fuera de la LODC (laboral, penal, arrendamientos, tributaria, familiar)** | El portero semántico descarta relatos ajenos al consumo | Producción: F17 y F18 rechazados; abstención en S14, S15, S16 (docs/08-pruebas.md) |
| **No inventa artículos** | Metadatos del corpus; sin fragmentos tras 3 reintentos pide aclaración; abstención 0,40 | 4/4 abstenciones; corpus LODC 64/64 verificado |
| **Datos de identificación (nombre, cédula, contacto) fuera del modelo** | Se piden solo al generar el documento y se inyectan por código | docs/03-arquitectura.md; riesgo residual R4 |

## B · Limitaciones técnicas

- **Relato libre:** puede traer datos personales y no se anonimiza antes del modelo (R4).
- **Cálculos:** solo tipos 1, 2 y 4; 3, 5, 6, 7 y 8 sin cálculo determinista.
- **Sin bucle ReAct:** capas con pasos fijos; el modelo no elige el siguiente paso.
- **Evaluación:** p95 solo en /start y con n = 20.
- **Despliegue:** plan gratuito (arranque en frío ~80 s), sin CI, sin modelo de respaldo.

*«Sabemos qué no debe hacer el sistema, y lo comprobamos con casos reales».*